МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 9

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил: доцент, доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы:

Узнать, как работает датчик температуры DS18B20 и как им пользоваться.

Задания на лабораторную работу: написать программу для использования датчика температуры.

1. Теоретические сведения

ROM-команды:

1) Search ROM [0xF0] – данная команда используется при первичной инициализации системы для получения идентификаторов устройств.

Любое 1-Wire устройство имеет 64-разрядный код. Этот код хранится в ПЗУ и используется для адресации устройства на шине. Младший байт кода определяет семейство, к которому принадлежит 1-Wire устройство (например, код датчиков DS18B20 – 0x28). Следующие 6 байт – это серийный номер устройства. И старший байт – CRC (циклический избыточный код), вычисленный из первых 7 байтов. Процедура получения адресов выглядит следующим образом. Выполнив инициализацию, микроконтроллер посылает команду Search ROM, и начинается цикл чтения 64-разрядного кода. Микроконтроллер формирует на 1-Wire шине два тайм-слота чтения. В первый тайм-слот все 1-Wire устройства, подключенные к шине, выдают первый бит своего 64-разрядного кода. Во второй тайм-слот – инвертированное значение первого бита. Если у всех 1-Wire устройств первый бит адреса – единица, то микроконтроллер примет сначала единицу, а затем ноль. Если хотя бы у одного 1-Wire устройства первый бит адреса нулевой, то микроконтроллер в обоих случаях примет ноль. Если активных устройств на шине нет, микроконтроллер в обоих случаях примет единицу. В ситуации, когда микроконтроллер принимает два нуля, возникает неоднозначность: как узнать, у каких устройств преданный бит адреса ноль, а у каких – единица? Протокол решает эту проблему просто: после двух тайм слотов чтения микроконтроллер должен ответить 1-Wire устройствам, с какими из них он продолжит работу. Для этого он выставляет на шине соответствующий бит: ноль или единицу. Устройства, у которых переданный бит соответствует выставленному микроконтроллером, продолжат работу, остальные замолчат (станут неактивными) до следующего сигнала сброса. Далее процедура

повторяется еще 63 раза: формирование первого тайм-слота чтения, чтение состояния шины, формирование второго тайм-слота чтения, чтение состояния шины, ответ подчиненным устройствам. После завершения цикла чтения 64-разрядного кода, микроконтроллер будет знать адрес одного 1-Wire устройства. Для получения следующего адреса нужно снова запустить цикл чтения, но на этот раз в случае неоднозначности выставить другой бит. Сколько 1-Wire устройств подключено к шине, столько раз и следует провести описанную процедуру.

2) Read ROM [0x33] – команда может использоваться только в случае, когда на шине всего одно подчиненное устройство. Позволяет напрямую получить его идентификатор.

3) Match ROM [0x55] – дает возможность мастеру выбрать конкретное устройство на шине: после отправки данной команды мастер отправляет идентификатор адресата, который и будет выполнять команду, которую затем отправит мастер.

4) Skip ROM [0xCC] – команда используется для адресации всех устройств на шине одновременно. Следующая за этой команда будет выполнена всеми подчиненными одновременно. Также может использоваться, если на шине всего одно подчиненное устройство.

5) Alarm Search [0xEC] – команда аналогична Search ROM, но отвечать будут лишь те устройства, у которых установлен Alarm флаг.

Функциональные команды:

1) Convert T [0x44] – запуск преобразования температуры. Если DS18B20 работает в режиме паразитного питания, то после выдачи команды Convert T микроконтроллер должен подключить 1-Wire шину к источнику питания на время преобразования. При работе от внешнего питания во время хода преобразования датчик удерживает на линии 0, по окончании выставляет 1.

2) Read Scratchpad [0xBE] – эта команда позволяет микроконтроллеру считывать содержимое ОЗУ датчика DS18B20. Данные передаются младшим битом вперед с нулевого по восьмой байт. Микроконтроллер может в любой момент выдать на 1-Wire шину сигнал сброса и прекратить прием данных, если, например, требуются только первые два байта ОЗУ.

3) Write Scratchpad [0x4E] – запись данных в память датчика: передается 3 байта, первый пишется в регистр TH, второй – в TL, третий – в регистр конфигурации.

4) Copy Scratchpad [0x48] – переписывает содержимое регистров TH, TL и регистра конфигурации в энергонезависимую память датчика. Если DS18B20 работает в режиме паразитного питания, то после выдачи команды Copy Scratchpad микроконтроллер должен подключить 1-Wire шину к источнику питания не менее чем на 10 мс.

5) Recall E2 [0xB8] – восстанавливает содержимое регистров TH, TL и регистра конфигурации из энергонезависимой памяти датчика. Микроконтроллер может отслеживать процесс выполнения этой операции, формируя на 1-Wire шине тайм-слоты чтения. Пока операция выполняется, на шине будет логический ноль, когда операция завершится – логическая единица. Выполняется автоматически при включении датчика.

6) Read Power Supply [0xB4] – используется мастером для определения, используется ли каким-либо из датчиков на шине паразитное питание.

2. Практическая часть

Код программы:

```
#define F_CPU 8000000UL
#define D4 PINB0
#define D5 PINB1
#define D6 PINB2
#define D7 PINB3
#define RS PINB5
#define RW PINB6
#define E PINB7
#define CMD 0
#define DATA 1
#define WIRE PIND4

#define SET_1_WIRE    PORTD|=(1 << WIRE);
#define SET_0_WIRE    PORTD&=~(1 << WIRE);

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void DisplayData (uint16_t data);
void InitADC(void);
void InitUSART(void);
void SendChar(char symbol);
void SendString(char * buffer);
void InitLCD(void);
void LCD_Write(uint8_t type, char data);
char LCD_Read(void);
void OneWire_Init(void);
uint8_t OneWire_Read_1_0(void);
uint8_t OneWire_ReadByte(void);
void OneWire_Send_1_0(uint8_t);
void OneWire_SendByte(uint8_t);

volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t ADC_val, temperature = 0;
const char char_tab[] =
{'0','1','2','3','4','5','6','7','8',
```

```
'9','A','B','C','D','E','F'};

int main(void)
{
	InitPorts();
	InitTimer1();
	EIMSK |= (1<<INT0);
	EICRA |= (1<<ISC01);
	InitADC();
	InitUSART();
	InitLCD();
	sei();
	SendString("Hello\r\n");
	LCD_Write(DATA, 'T');
	LCD_Write(DATA, '=');
	LCD_Write(DATA, 0x20);
	LCD_Write(CMD, 0x40 | 0x80);
	LCD_Write(DATA, 'V');
	LCD_Write(DATA, 'a');
	LCD_Write(DATA, 'l');
	LCD_Write(DATA, 'u');
	LCD_Write(DATA, 'e');
	LCD_Write(DATA, '=');
	LCD_Write(DATA, 0x20);
	OneWire_Init();
	OneWire_SendByte(0xCC); // Skip ROM
	OneWire_SendByte(0x4E); // Write scratchpad
	OneWire_SendByte(0x00); // TH
	OneWire_SendByte(0x00); // TL
	OneWire_SendByte(0x3F); // config

	while(1)
	{
		Bin2Dec(ADC_val);
		LCD_Write(CMD, 0x47 | 0x80);
		LCD_Write(DATA, 0x30+bcd_buffer[3]);
		LCD_Write(DATA, 0x30+bcd_buffer[2]);
		LCD_Write(DATA, 0x30+bcd_buffer[1]);
		LCD_Write(DATA, 0x30+bcd_buffer[0]);

		OneWire_Init();
		OneWire_SendByte(0xCC); // Skip ROM
		OneWire_SendByte(0x44); // Convert T
		while (OneWire_Read_1_0() == 0);
```

```
		OneWire_Init();
		OneWire_SendByte(0xCC); // Skip ROM
		OneWire_SendByte(0xBE); // Read Scratchpad
		temperature = 0xFF & OneWire_ReadByte();
		temperature |= OneWire_ReadByte() << 8;

		LCD_Write(CMD, 0x03 | 0x80);
		LCD_Write(DATA, char_tab[(temperature >> 12) & 0x0F]);
		LCD_Write(DATA, char_tab[(temperature >> 8) & 0x0F]);
		LCD_Write(DATA, char_tab[(temperature >> 4) & 0x0F]);
		LCD_Write(DATA, char_tab[(temperature >> 0) & 0x0F]);
	}
}
//-------------------------------------------

ISR(TIMER1_COMPB_vect){
	//DisplayData(0x1E61);
}

ISR(INT0_vect){
	Bin2Dec(ADC_val);
	SendString("Value = ");
	SendChar(0x30 + bcd_buffer[3]);
	SendChar(0x30 + bcd_buffer[2]);
	SendChar(0x30 + bcd_buffer[1]);
	SendChar(0x30 + bcd_buffer[0]);
	SendString("\r\n");
}
ISR(ADC_vect){
	ADC_val = ADC;
}
ISR(USART_RX_vect){
	if(UDR0 == 0x20){
		SendString("Roger that\r\n");
	}
}
//-------------------------------------------
void InitPorts(void){
	DDRB = 0xFF; //настройка выводов управления дисплеем
	PORTB = 0;
	DDRD = (0 << PIND2 | 1 << PIND4); //настройка вывода кнопки
	PORTD |= (1 << PIND2 | 1 << PIND4);
}
void InitTimer1( void){
	TCCR1A = 0;
```

```
        TCCR1B = (1 << CS11 | 1 << CS10 | 1 << WGM12);
        TCNT1 = 0;
        TIMSK1 |= (1 << OCIE1B);
        OCR1A = 12500;
        OCR1B = 12500;
}
void Bin2Dec(uint16_t data){
        bcd_buffer[3] = (uint8_t)(data/1000);
        data = data % 1000;
        bcd_buffer[2] = (uint8_t)(data/100);
        data = data % 100;
        bcd_buffer[1] = (uint8_t)(data/10);
        data = data % 10;
        bcd_buffer[0] = (uint8_t)(data);
}
void DisplayData (uint16_t data){
        Bin2Dec(data);
}
void InitADC( void){
        ADMUX = (1 << MUX0);
        //Align left, ADC1
        ADCSRB = (1 << ADTS2 | 1 << ADTS0); //Start on Timer1 COMPB
        //Enable, auto update, IRQ enable
        ADCSRA = (1 << ADEN | 1 << ADATE | 1 << ADIE);
}
void InitUSART(){
        UCSR0B = (1 << RXEN0 | 1 << TXEN0 | 1 << RXCIE0);
        UCSR0C = (1 << UCSZ01 | 1 << UCSZ00);
        UBRR0H = 0;
        UBRR0L = 0x67;
}
void SendChar(char symbol){
        while (!(UCSR0A & (1<<UDRE0)));
        UDR0 = symbol;
}
void SendString(char * buffer){
        while(*buffer != 0){
                SendChar(*buffer++);
        }
}

void InitLCD( void){
        uint8_t BF = 0x80;
        _delay_ms(40);
        PORTB &= ~(1 << RS);
```

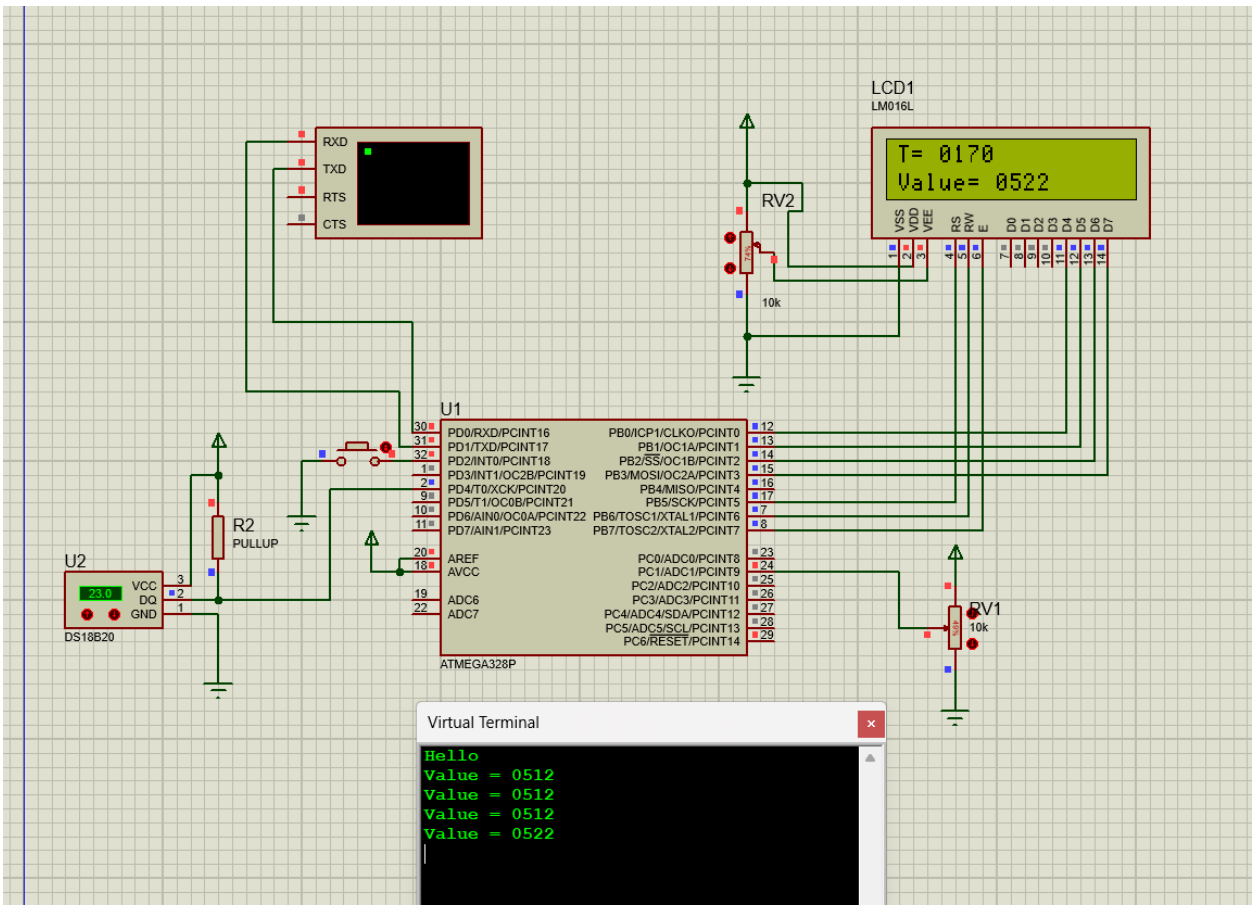
```
        PORTB = (0x30 >> PINB4);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB &= ~(1 << E);
        PORTB = 0;
        _delay_ms(5);
        PORTB &= ~(1 << RS);
        PORTB = (0x30 >> PINB4);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB &= ~(1 << E);
        PORTB = 0;
        _delay_us(150);
        PORTB &= ~(1 << RS);
        PORTB = (0x30 >> PINB4);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB &= ~(1 << E);
        PORTB = 0;
        _delay_ms(5);
        do{
                BF = (0x80 & LCD_Read());
        } while (BF == 0x80);
        PORTB &= ~(1 << RS);
        PORTB = (0x20 >> PINB4);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB &= ~(1 << E);
        PORTB = 0;
        do{
                BF = (0x80 & LCD_Read());
        } while (BF == 0x80);
        // заработал 4-проводной интерфейс
        LCD_Write(CMD, 0x28); //2 строки, 5*8
        LCD_Write(CMD, 0x0C); //display on, cursor on
        LCD_Write(CMD, 0x06); //cnt++, shift enabled
}
```

```
void LCD_Write(uint8_t type, char data){
	uint8_t BF=0x80;
	do{
		BF = 0x80 & LCD_Read();
	} while (BF == 0x80);
	PORTB |= (type << RS);
	PORTB |= (1 << E);
	PORTB &= ~(0x0F);
	PORTB |= (0x0F & (data >> PINB4)); //старшая тетрада
	PORTB &= ~(1 << E);
	asm("nop");
	asm("nop");
	asm("nop");
	PORTB |= (1 << E);
	PORTB &= ~(0x0F);
	PORTB |= (0x0F & data); //младшая тетрада
	PORTB &= ~(1 << E);
	PORTB = 0;
}
char LCD_Read( void){
	char retval=0;
	PORTB &= ~(1 << RS);
	PORTB |= (1 << RW);
	DDRB &= ~(1 << D4 | 1 << D5 | 1 << D6 | 1 << D7);
	PORTB |= (1 << E);
	asm("nop");
	asm("nop");
	retval = (PINB & 0x0F) << 4;
	PORTB &= ~(1 << E);
	asm("nop");
	asm("nop");
	asm("nop");
	PORTB |= (1 << E);
	asm("nop");
	asm("nop");
	retval |= (PINB & 0x0F);
	PORTB &= ~(1 << E);
	DDRB |= (1 << D4 | 1 << D5 | 1 << D6 | 1 << D7);
	PORTB = 0;
	return retval;
}
void OneWire_Init( void){
	DDRD |= (1 << WIRE); // шину на выход
	SET_0_WIRE;	// установить 0
	_delay_us(500);
```

```
        // ждать не менее 480 мкс
        SET_1_WIRE;  // шину на вход
        DDRD&= ~(1 << WIRE);
        _delay_us(60);
        if ((PIND & (1 << WIRE)) == 0)
        { // ожидается импульс сброса
                while ((PIND & (1 << WIRE)) == 0);
                _delay_us(420);
        }
        else
        {
                SendString("No response\r\n");
        }
}
void OneWire_Send_1_0(uint8_t bit){
        _delay_us(2);
        DDRD |= (1 << WIRE);
        SET_0_WIRE;
        _delay_us(10);
        if (bit != 0)
        {
                SET_1_WIRE;
                _delay_us(90);
        }
        else
        {
                SET_0_WIRE;
                _delay_us(90);
                SET_1_WIRE
        }
}
uint8_t OneWire_Read_1_0(void){
        uint8_t bit = 0;
        _delay_us(2);
        DDRD |= (1 << WIRE);
        SET_0_WIRE;
        _delay_us(2);
        SET_1_WIRE;
        DDRD &= ~(1 << WIRE);
        _delay_us(2);
        if((PIND & (1 << WIRE)) == 0)
        {
                bit = 0;
        }
        else
```

```
	{
		bit = 1;
	}
	_delay_us(90);
	return bit;
}
void OneWire_SendByte(uint8_t data){
	for (uint8_t i=0; i<8; i++)
	{
		OneWire_Send_1_0(0x01 & data);
		data = data >> 1;
	}
}
uint8_t OneWire_ReadByte( void){
	uint8_t retval = 0;
	for (uint8_t i=0; i<8; i++)
	{
		retval |= OneWire_Read_1_0() << i;
	}
	return retval;
}
```

Схема в Proteus:


LCD1
LM016L
T= 0170
Value= 0522
RV2
10k
RXD
TXD
RTS
CTS
U1
ATMEGA328P
R2
PULLUP
U2
DS18B20
RV1
10k
Virtual Terminal
Hello
Value = 0512
Value = 0512
Value = 0512
Value = 0522

Вопросы:

1)Назовите преимущества цифровых датчиков температуры по сравнению с аналоговыми.

При цифровой передаче данных достигается большая помехоустойчивость, легко организуется гальваническая развязка сигнала.

2)Что такое «паразитное питание» для датчика DS18B20?

Теоретически необходимы три провода, провод питания, провод для линии данных и земля. Однако есть возможность использовать всего два провода, избавившись от провода питания. Датчик получает энергию от линии передачи данных: в то время, пока не идет обмена данными, датчик заряжает от линии встроенный конденсатор, который играет роль источника питания в те моменты, когда происходит обмен данными, такая технология и получила название «паразитное питание».

3)Интерфейс 1-Wire позволяет подключать к одной шине множество подчиненных устройств. Как происходит обращение к конкретному устройству?

Обращение к конкретному устройству по одной шине 1-Wire происходит по идентификатору устройства. У каждого датчика DS18B20 собственный уникальный 64-битный идентификатор, записанный в его память.

4)Опишите общую последовательность обмена данными по шине 1-Wire.

Любой обмен по шине 1-Wire происходит в последовательности:

1) Процедура инициализации
2) Передача ROM-команды с необходимой последующей информацией
3) Передача функциональной команды с необходимой информацией.

При инициализации, мастер отправляет импульс сброса, на который подчиненные отвечают импульсами присутствия. Микроконтроллер на 480

мкс устанавливает на 1-Wire шине ноль, а затем освобождает ее. Если к шине подключен термометр, то он обнаруживает положительный перепад и после паузы в 15-60 мкс отвечает микроконтроллеру импульсом присутствия, т.е. устанавливает на шине ноль на время от 60 до 240 мкс.

После инициализации, мастер подает код ROM-команды, которые направлены на работу с идентификаторами датчиков и служат для выбора конкретного устройства из множества подключенных. Эти команда позволяют мастеру определить, сколько и какие устройства подключены к шине. Всего есть 5 ROM-команд, каждая из которых состоит из 1 байта.

После выбора необходимых подчиненных устройств с помощью ROM-команды мастер передает функциональную команду, которая определяет действия подчиненного. Всего имеется 6 функциональных команд.

Выводы

Я изучил принцип работы датчика температуры и научился пользоваться им.